

프로젝트 관리론

Chapter 03 계획

목차

01 계획의 이해

02 문제 정의

03 타당성 분석

04 개발 비용 산정

05 비용 산정 기법 1: 하향식 산정 기법

06 비용 산정 기법 2: 상향식 산정 기법

07 비용 산정 기법 3: 수학적 산정 기법

08 일정 계획

09 위험 분석

학습목표

- 체계적인 계획을 세우는 방법을 알아본다.
- 소프트웨어 개발 비용 산정을 알아본다.
- 기능 점수 방법을 자세히 살펴본다.
- 프로젝트 일정 계획과 위험 분석 방법을 알아본다.



Section 01 계획의 이해

1. 계획의 이해



그림 3-1 초등학교생의 방학 중 하루 일과표와 개학 전의 모습

2. 소프트웨어 개발 계획

■ 소프트웨어 개발 계획

- 비용, 기간, 자원 계획 필요

■ 계획 없는 소프트웨어 개발

- 일정 지연, 비용 초과, 품질 저하 → 유지보수 비용 증가



그림 3-2 프로젝트 마감일을 앞둔 프로젝트 팀의 모습

2. 소프트웨어 개발 계획

- 소프트웨어 개발 계획 세우는 방법 및 순서

- 문제 정의
- 타당성 분석
- SW 개발 비용 산정
- SW 개발 비용 산정 기법
- 일정 계획
- 위험 분석



Section 02 문제의 정의

1. 문제의 정의

■ 문제 정의

- 소프트웨어 개발의 첫 작업
- 무엇을 개발할 것인지 명확히 정의
- 개발 범위를 결정

→ 프로젝트의 초기 타당성과 초기 계획을 작성할 수 있는 기초로 활용

■ 문제 정의를 위한 필요 사항

- 개발하고자 하는 영역의 배경 지식 필요하다.
- 문제를 파악하기 위해 현재 운영 중인 시스템을 사용해본다.
- 실무 담당자와 면담하여 자료를 수집한 후 면밀히 분석해본다.



Section 03 타당성 분석

1. 타당성 분석(1)

■ 경제적 타당성economic feasibility

- 경영자: 투자 효율성cost benefit analysis에 관심
- 분석가: 투자 대비 효과 검토 후 경영자에게 정확한 정보 제공
- 시장 분석을 통한 시장성marketability 확인

→ 개발 여부 판단

■ 기술적 타당성technical feasibility

- 현재의 기술로 사용자가 요구하는 기능을 구현할 수 있는지 검토
- 하드웨어 성능이 개발에 지장은 없는지 검토
- 개발자의 기술력에 문제가 없는지 검토

1. 타당성 분석(2)

■ 법적 타당성 legal feasibility

- 개발용 소프트웨어와 도구의 사용이 법적으로 문제가 없는지 검토
- 지적 소유권과 프로그램 보호법이 강화되었으므로 법적인 문제를 꼭 검토



Section 04 개발비용 산정

1. 개발 비용 산정



2. 개발비 산정의 어려움(1)

■ 전자 제품 생산 비용 예측

- 생산 제품 형상의 명확화
- 생산에 투입되는 자재 개수 및 가격의 명확화

→ 제조 원가의 명확화

조립 PC 기본 세트						☞ 찾기
CPU	인텔 코어5-2세대 2500K (샌디브릿지) (정품)	180	1	232,000	₩	상세
메인보드	ASUS P6P67 PRO (B3) STCOM	80	1	244,000	₩	상세
RAM	삼성전자 DDR3 4G PC3-10600 (정품)	196	2	24,000	₩	상세
VGA	MSI 지포스 GTX560 Ti N560GTX-Ti O C D5 1GB 트윈프로저 2	190	1	277,500	₩	상세
SSD	삼성전자 S470 Series (64GB, MZ-5PA 064/KR, 정품)	87	1	140,000	₩	상세
HDD	WD 500GB Caviar Blue WD5000AAKX (SATA3/7200/16M)	206	1	39,800	₩	상세
ODD	삼성전자 Super-WriteMaster SH-222A L (블랙 정품벌크)	153	1	21,890	₩	상세

■ 건축 비용 예측

- 자재비, 인건비의 정확한 예측 가능
- 전체 비용 정확한 산출(평당 가격 X 평수)

구분	품목	규격 (피트×인치×인치)	수량(본)	단가(원)	금액(원)
방부목 (Green) 일 경우	기둥	6×6×6	8	22,464	179,712
	통배리목	2×6×6	8	7,488	59,904
	핸드레일류	12×4×2	21	8,463	177,723
	핸드레일류	4×2×2	100	1,274	127,400
	측면/계단판류	12×12×2	13	30,030	390,390
	장선 양어 각재	12×6×2	20	13,104	262,080
	바닥재	12×6×1	120	8,237	988,416
	화이트 레티스	8×4(10)트	7(평)	45,000	315,000
자재비 소계			10	30,000	300,000
					2,800,625

2. 개발비 산정의 어려움(2)

■ 소프트웨어 개발 비용 예측

- 가전제품 생산이나 건축 공사와 달리 사람(개발자)이 중심
- 개발자의 능력에 따른 생산성의 차이 → 기간과 품질에 영향
- 다양한 개발 프로세스로 인한 표준화/자동화의 어려움 → 다양한 생산성과 품질

→ 명확한 개발비 산출의 어려움

3. SW 개발 비용에 영향을 주는 요소(1)

■ 프로그램 자질

- 초급 프로그래머와 고급 프로그래머의 생산성 차이가 큼
→ 프로그램의 자질: 개발 비용에 영향을 줌

■ 소프트웨어 복잡도

- 브룩스의 법칙: 애플리케이션 개발 < 유틸리티 개발 < 시스템 프로그램 개발
→ 소프트웨어 복잡도: 개발 비용에 영향을 미침

■ 소프트웨어 크기

- 참여 인력 증가, 개발 기간 길어짐, 복잡도 커짐
→ 소프트웨어 규모: 개발 비용에 영향을 미침

3. SW 개발 비용에 영향을 주는 요소(2)

■ 가용 시간

- 관리자들의 잘못된 생각: 인력/자원 증가는 개발 기간 단축
- 보헴: "정상적인 계획에서 최대 75%가 줄일 수 있는 한계"

■ 요구되는 신뢰도 수준

- 높은 신뢰도의 소프트웨어 개발: 개발 비용의 증가

■ 기술 수준

- 고급언어 사용: 저급 언어의 사용보다 5~10배의 생산성 증가



Section 05 비용 산정 기법 1 : 하향식 산정 기법

1. 하향식 산정 기법

■ 전문가 판단 기법

- 경험이 많은 전문가가 개발 비용을 산정 → 신뢰성 높음
- 짧은 시간에 개발비를 산정하거나 입찰에 응해야 하는 경우 많이 사용
- 단점: 수학적 계산 방법보다 경험에만 의존할 경우 부정확할 수 있음



■ 델파이 기법

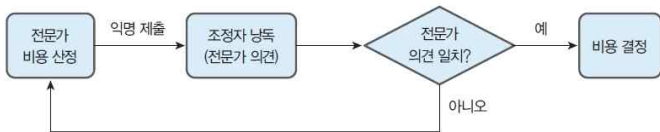


그림 3-3 델파이 기법의 비용 산정 방법



Section 06 비용 산정 기법 2 : 상향식 산정 기법

1. 상향식 산정 기법

■ 상향식 산정 기법

- 세부 작업 단위별로 비용 산정한 후 전체 비용 합산
- 단점: 수학적 계산 방법보다 경험에만 의존할 경우 부정확할 수 있음

① 원시 코드 라인 수^{LOC} 기법

- 원시 코드 라인 수의 비관치, 낙관치, 중간치를 측정 후 예측치를 구해 비용 산정

② 개발 단계별 노력^{effort per task} 기법

- 생명주기의 각 단계별로 노력(M/M)을 산정

2. 원시 코드 라인 수 기법

■ LOC 기법

비관/낙관/기대치 측정

→ 예측치 계산

→ 비용 산정(노력, 개발 비용, 생산성)

- 낙관적 : 한 모듈의 라인 수를 가장 적게 생각할 때의 예상 라인 수(가중치 0 부여)
- 비관적 : 한 모듈의 라인 수를 가장 많게 생각할 때의 예상 라인 수(가중치 1 부여)
- 보통 : 한 모듈의 라인 수를 보통이라고 생각할 때의 예상 라인 수(가중치 4 부여)
- 추정 LOC : (낙관치+(4X중간치)+비관치)/6

- 노력(인/월수 M/M) = (참여 인원/월) * 개발기간 = 추정 LOC/1인당 월 평균 생산 코드 라인 수
- 개발 비용 = (M/M) * 단위 비용(1인당 월 평균 인건비)
- 개발 기간 = (M/M) / 참여 인원
- 생산성 = LOC / (M/M)

2. 원시 코드 라인 수 기법

- 노력(인/월수 M/M) = (참여 인원/월) * 개발기간 = 추정 LOC/1인당 월 평균 생산 코드 라인 수
- 개발 비용 = (M/M) * 단위 비용(1인당 월 평균 인건비)
- 개발 기간 = $(M/M) /$ 참여 인원
- 생산성 = $LOC / (M/M)$

- 소프트웨어 개발 기간은 1년(12개월)이다. 5명의 개발자가 12개월 동안, 7명의 개발자가 5개월 동안 참여한다면 이 소프트웨어의 개발의 노력 M/M 은 얼마인가?
- LOC 기법에 의해 예측한 총 라인이 50,000라인이고, 개발자가 10명 참여한다. 그리고 개발자들이 월 평균 500라인을 코딩한다면 개발 기간은 얼마나 되는가?

3. 개발 단계별 노력(M/M)기법

■ LOC

- 개발하려는 소프트웨어의 총 코드 라인 수를 예측하여 구현 단계에 대한 M /M을 산정

■ 실제 소프트웨어 개발

- 코딩뿐 아니라 요구 분석, 설계 등의 단계에서도 인력과 자원이 많이 필요



LOC 보완

■ 개발 단계별 노력 기법

- M/M을 소프트웨어 개발 생명주기의 각 단계에 적용하여 단계별로 산정
- 장점: 코딩만 대상으로 산정하는 LOC보다 더 정확



Section 07 비용 산정 기법 3 : 수학적 산정 기법

1. 수학적 산정 기법

- 상향식 비용 산정 기법
- 경험적 추정 기법 또는 실험적 추정 기법

① COCOMO 방법

- SW 규모(LOC) 예측한 후 SW 종류에 따라 각 비용 산정 공식에 대입하여 비용 산정

② Putnam 방법

- 소프트웨어 생명주기의 전 과정에 사용될 노력의 분포를 **가정해 줌**

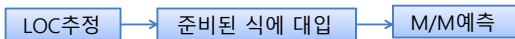
③ 기능 점수(FP) 방법

- 기능 점수를 구한 후 이를 이용해 비용 산정

2. COCOMO 방법(1)

■ COCOMO 방법

- 라인수 중심의 개발비 산정



■ 개발비 산정 시 고려 사항

- 프로그램 유형(난이도)에 따른 가중치

표 3-1 투입 인력 산출 공식과 프로젝트 유형에 따른 상수 a, b 값

$$PM = a \times (KDSI)^b$$

상수	유형별 값
a	단순형(2.4), 중간형(3.0), 내장형(3.6)
b	단순형(1.05), 중간형(1.12), 내장형(1.20)

2. COCOMO 방법(2)

■ 네 가지 특성에 따른 15가지 분류

표 3-3 COCOMO 방법에서 사용되는 노력 승수 값

특성	비용 승수 요소	승수 값					
		매우 낮음	낮음	정상	높음	매우 높음	극히 높음
제품 특성	요구되는 신뢰도	0.75	0.88	1.00	1.15	1.40	
	데이터베이스 크기	0.94	1.00	1.08	1.16		
	제품의 복잡도	0.70	0.85	1.00	1.15	1.30	1.65
컴퓨터 특성	실행 시간 제약			1.00	1.11	1.30	1.66
	주기억 장치의 제약			1.00	1.06	1.21	1.56
	HW/SW의 안정성		0.87	1.00	1.15	1.30	
	처리 시간		0.87	1.00	1.07	1.15	
개발자 특성	분석가의 능력	1.46	1.19	1.00	0.86	0.71	
	응용 분야 경험	1.29	1.13	1.00	0.91	0.82	
	컴퓨터와 친숙성	1.21	1.10	1.00	0.90	-	
	프로그래머 능력	1.42	1.17	1.00	0.86	0.70	
	프로그래밍 언어의 경험	1.14	1.07	1.00	0.95		
프로젝트 특성	소프트웨어 공학 기술 사용	1.24	1.10	1.00	0.91	0.82	
	소프트웨어 도구의 사용	1.24	1.10	1.00	0.91	0.83	
	요구되는 개발 일정	1.23	1.08	1.00	1.04	1.10	

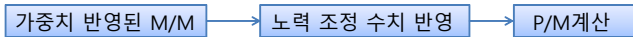
2-1 COCOMO 방법을 이용한 총 개발 기간 산정 과정

① 가중치 반영하기

- 프로그램의 규모 LOC에 가중치가 반영된 초기 개발 인건비 M/M 산정



② 보정 계수 반영 하기



③ 총 개발 기간 산정하기



2-2 프로젝트 유형

■ 단순형 프로젝트

- 복잡도와 난이도가 비교적 높지 않은 업무용 소프트웨어
- 크기는 중소 규모 정도, 크기는 50KDSI 이하

■ 중간형 프로젝트

- 규모나 복잡도가 중간급 정도, 크기 300KDSI 이하
- 운영체제, 데이터베이스 관리 프로그램

■ 내장형 프로젝트

- 자동화 기기, 전자 제품과 같은 하드웨어와 밀접하게 관련 있는 소프트웨어
- 크기 300KDSI 이상

2-3 가중치 반영하기(1)

■ 개발 인건비의 초기 예측 값 산출 방법

표 3-2 프로젝트 유형에 따른 투입 인력 산출 공식

프로젝트 유형	공식
단순형	$PM = 2.4 \times (KDSI)^{1.05}$
중간형	$PM = 3.0 \times (KDSI)^{1.12}$
내장형	$PM = 3.6 \times (KDSI)^{1.20}$

- $PM^{Person/Month}$: 소프트웨어 개발에 필요한 인력(인원/월)
- $KDSI^{Kilo\ Delivered\ Source\ Instruction}$: 소프트웨어의 최종 원시 코드 라인 수

2-3 가중치 반영하기(2)

■ 유형별 M/M

- 단순형 < 중간형 < 내장형

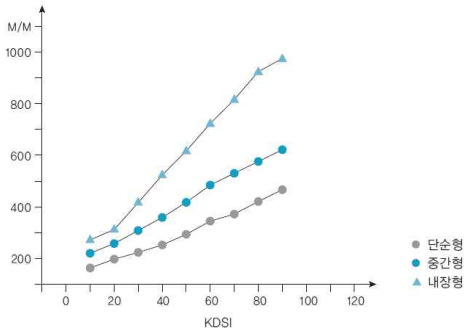


그림 3-4 COCOMO 방법에 의한 비용 예측

2-4 보정 계수 반영하기(1)

■ 노력 조정 수치 EAF: Effort Adjustment Factor

- 보정에 사용되는 값

노력조정 수치^{EAF} = 필요한 각 항목에 승수 값을 모두 곱한 값

(예)

개발하려는 소프트웨어에 요구되는 신뢰도가 높고(1.15), 매우 복잡하며(1.30), 소프트웨어공학 기술을 거의 사용하지 않고(1.24), 요구되는 개발 일정이 매우 촉박하고(1.10), 다른 요소들은 보통(1.00)일 경우의 노력 조정 수치?

$$EAF = 1.15 \times 1.30 \times 1.24 \times 1.10 = 2.04$$

가중치 반영된 M/M

노력 조정 수치 반영

P/M계산

2-4 보정 계수 반영하기(2)

표 3-3 COCOMO 방법에서 사용되는 노력 승수 값

특성	비용 승수 요소	승수 값					
		매우 낮음	낮음	정상	높음	매우 높음	극히 높음
제품 특성	요구되는 신뢰도	0.75	0.88	1.00	1.15	1.40	
	데이터베이스 크기	0.94	1.00	1.08	1.16		
	제품의 복잡도	0.70	0.85	1.00	1.15	1.30	1.65
컴퓨터 특성	실행 시간 제약			1.00	1.11	1.30	1.66
	주기억 장치의 제약			1.00	1.06	1.21	1.56
	HW/SW의 안정성		0.87	1.00	1.15	1.30	
	처리 시간		0.87	1.00	1.07	1.15	
개발자 특성	분석가의 능력	1.46	1.19	1.00	0.86	0.71	
	응용 분야 경험	1.29	1.13	1.00	0.91	0.82	
	컴퓨터와 친숙성	1.21	1.10	1.00	0.90	-	
	프로그래머 능력	1.42	1.17	1.00	0.86	0.70	
	프로그램 언어의 경험	1.14	1.07	1.00	0.95		
프로젝트 특성	소프트웨어 공학 기술 사용	1.24	1.10	1.00	0.91	0.82	
	소프트웨어 도구의 사용	1.24	1.10	1.00	0.91	0.83	
	요구되는 개발 일정	1.23	1.08	1.00	1.04	1.10	

2-4 보정 계수 반영하기(3)

■ 노력 조정 수치가 반영된 노력(P/M)

표 3-4 노력 조정 수치가 반영된 유형별 노력^{P/M}

프로젝트 유형	공식
단순형	$PM = 2.4 \times (KDSI)^{1.05} \times EAF$
중간형	$PM = 3.0 \times (KDSI)^{1.12} \times EAF$
내장형	$PM = 3.6 \times (KDSI)^{1.20} \times EAF$

(예)

만일 개발하려는 소프트웨어의 KDSI가 60이고, 소프트웨어는 중간형이며, 노력 조정 수치(EAF)가 2.04라면 노력(E)?

$$PM = 3.0 \times (KDSI)^{1.12} \times EAF = 3.0 \times (60)^{1.12} \times 2.04 = 600.179$$

2-5 총 개발 기간 산정하기

■ 총 개발 기간 TDEV: Total DEvelopment time

표 3-5 총 개발 기간 산정 공식

프로젝트 유형	공식
단순형	$TDEV = 2.5 \times (PM)^{0.38}$
중간형	$TDEV = 2.5 \times (PM)^{0.35}$
내장형	$TDEV = 2.5 \times (PM)^{0.32}$

(예)

만일 개발하려는 소프트웨어의 KDSI가 60이고, 소프트웨어는 중간형이며, 노력 조정 수치(EAF)가 2.04라면 노력(E)=600.179이다. 이 때 총 개발 기간은?

$$TDEV = 2.5 \times (PM)^{0.35} = 2.5 \times (600.179)^{0.35} = 23.461$$

3. COCOMO II 방법

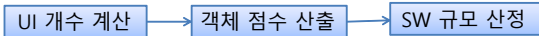
COCOMO의 문제점: (계획 단계)원시코드 라인수 예측



COCOMOII: 단계별로 값을 예측한 후 인건비 예측

3-1 프로젝트 진행 정도에 따른 분류

[1단계] 애플리케이션 합성 모델 application composition model



[2단계] 초기 설계 모델 early design model

- 초기 설계 단계에서 예측 값을 구한다.
- 1단계보다 더욱 정확한 예측이 가능

[3단계] 구조 설계 이후 모델 post-architecture model

- 기능 점수를 바탕으로 한 LOC를 추정하여 소프트웨어 규모를 산정
- $E = b \times S^C \times m(X)$
 $b \times S^C$: 기초 소요 노력 예측 값 $m(X)$: 비용 승수의 벡터

4. 기능 점수 산정 방법(1)

■ 산정 근거

- 기능(입·출력, 데이터베이스 테이블, 인터페이스, 조회 등)의 수
→ 라인 수와 무관하게 기능이 많으면 규모도 크고 복잡도도 높다고 판단
- 사용자관점에서 소프트웨어의 기능 정량화 => 개발 비용 산정

■ 기능 점수 function point

- 소프트웨어 기능의 크기를 측정하는 단위
(즉) 소프트웨어의 기능이 얼마나 복잡한가를 상대적인 점수로 표현하는 것

4. 기능 점수 산정 방법(2)

■ 용도

- 개발 시 비용 산정
- 유지보수 비용 산정
- 개발 시 필요한 자원 산정

■ 특징

- 소프트웨어 규모를 측정하는 방법
- 기능적 요구 사항이 중심이 되는 측정 방법
- 소프트웨어의 요구 사항 복잡도를 측정
- 구현 관점 아닌 사용자 관점의 요구 기능을 정량적으로 산정
- 측정의 일관성 유지를 위해 개발 기술, 개발 방법, 품질 수준 등은 고려하지 않음
- 소프트웨어 개발에 사용되는 언어와 무관
- 소프트웨어 개발 생명주기의 전체 단계에서 사용 가능

5. 소프트웨어 기능 분류

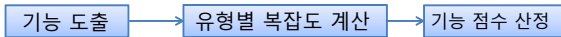


그림 3-5 소프트웨어 기능 분류

6. 기능 점수 산정 방법

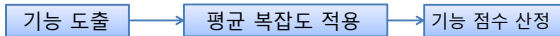
■ 정규 기능 점수법

- 설계 단계 이후에 사용하면 유용



■ 간이 기능 점수법

- 기획 및 발주 단계에서 사용



7. 기능 점수 산정 방법의 장점

- **사용자의 요구 사항만으로 기능을 추출하여 측정**
 - 실제 구현 방법과 무관
- **객관적인 요구 사항만으로 측정**
 - 개발 방법이나 개발 팀과 무관
- **모든 개발 단계에서 사용**
 - 계획 단계뿐 아니라, 분석, 설계, 구현 단계에서도 사용 가능

8. 기능 점수 산정 방법의 단점

- 높은 분석 능력 필요

- 요구 사항으로부터 기능을 도출하려면 상당한 분석 능력이 필요

- 기능 점수 전문가 필요

- 이 방법을 잘 사용할 수 있는 기능 점수 전문가가 필요

- 내부 로직 위주의 소프트웨어에는 다소 부적합

- 사용자가 알 수 있는 기능으로 측정하기 때문에 내부 로직 위주의 SW 측정에는 부적합

- 개발 규모 측정에 적합

- 개발 규모를 예측하는 데 적합

9. 간이 기능 점수법 산정

■ 평균 복잡도 가중치

- 5가지 유형에 적용된 복잡도에 대해 계산한 가중치의 평균값
 - 5가지 유형: 내부 논리 파일, 외부 연계 파일, 외부 입력, 외부 출력, 외부 조회

표 3-6 평균 복잡도 가중치

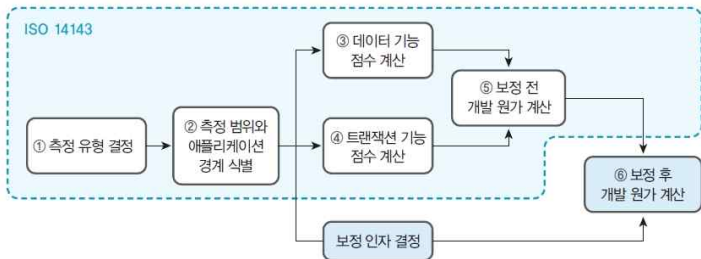
유형	내부 논리 파일	외부 연계 파일	외부 입력	외부 출력	외부 조회
가중치	7.5	5.4	4.0	5.2	3.9

출처 : SW사업 대가산정 가이드(2015년 개정판), 한국소프트웨어산업협회

■ 간이 기능 점수법

- 프로젝트 초기 단계에서 평균 복잡도 가중치를 사용하여 소프트웨어 기능의 크기를 측정

10. 간이 기능 점수 산정 절차



- ① 측정 유형(개발, 개선, 애플리케이션) 결정
- ② 개발하려는 소프트웨어의 측정 범위와 애플리케이션 경계 설정
- ③ 데이터 기능 점수 계산: 데이터 기능(내부 논리 파일, 외부 연계 파일) 도출 → 도출된 데이터 기능(내부 논리/외부 연계 파일)의 개수 × 평균복잡도 가중치
- ④ 트랜잭션 기능 점수 계산: 트랜잭션 기능(외부 입·출력, 외부 조회) 도출 → 도출된 트랜잭션 기능(외부 입·출력, 외부 조회)의 개수 × 평균복잡도 가중치
- ⑤ 보정 전 개발 원가 계산: 총 기능 점수 × 기능 점수당 단가(총 기능 점수 = 데이터 기능 점수 + 트랜잭션 기능 점수)
- ⑥ 보정 후 개발 원가 계산: 보정 전 개발 원가 × 4가지 보정 계수(규모, 애플리케이션 유형, 언어, 품질 및 특성)

그림 3-7 간이 기능 점수 산정 절차

출처: CPM 4.1, IFPUG, 1999

10-1 측정 유형 결정

■ 개발 프로젝트 기능 점수(개발 규모 측정)

- 프로젝트가 완료되어 최초 설치했을 때의 기능 크기를 산정
(즉) 프로젝트에서 사용자를 위해 제공된 모든 기능을 측정

■ 개선(enhancement) 프로젝트 기능 점수(변경 규모 측정)

- 사용 중인 소프트웨어에 변경이 발생했을 때 변경된 부분의 기능을 측정
(즉) 완료된 프로젝트에서 추가, 수정, 삭제된 부분만 그 크기를 측정

■ 애플리케이션 기능 점수(응용 규모 측정)

- 현재 운용 중인 애플리케이션의 기능을 측정
- 개발프로젝트 기능 점수에 개선 프로젝트 기능 점수까지 포함된 것

10-2 측정 범위와 애플리케이션 경계 식별(1)

신규로 개발되는 프로젝트를 대상으로 기능 점수를 측정하는 과정

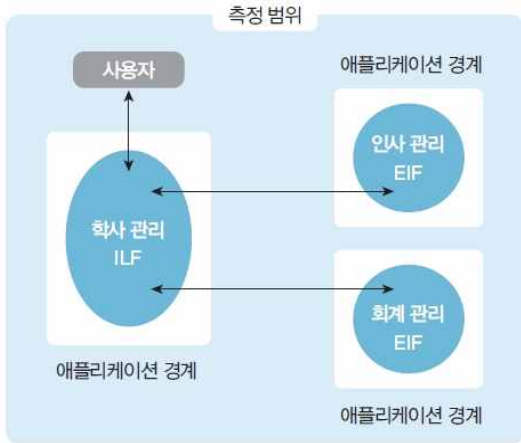


그림 3-8 대학 종합정보시스템의 애플리케이션 경계

10-2 측정 범위와 애플리케이션 경계 식별(2)

■ 측정 범위에 포함될 요소(서브시스템)의 식별

- 측정 범위: 기능 점수를 측정하고자 하는 대상
- 측정 대상: 개발하고자 하는 전체 소프트웨어 또는 소프트웨어의 일부 시스템

■ 애플리케이션 경계

- 금번 기능 점수를 계산하는 대상과 다른 애플리케이션이나 외부 사용자를 구분하는 경계

■ 애플리케이션 경계 식별 시 주의 점

- 사용자 관점에서 경계 구분
- 개발자 관점(클라이언트/서버, 서버/비즈니스 로직/클라이언트 계층)으로 구분하면 안됨
- 애플리케이션의 경계의 크기가 적당해야 함

10-3 데이터 기능 점수 계산(1)

■ 데이터 기능 점수 계산

- 내부 논리 파일(ILF)의 개수와 외부 연계 파일(EIF)의 개수를 계산하여 각각의 평균 복잡도(가중치)에 따라 데이터 기능 점수 결정

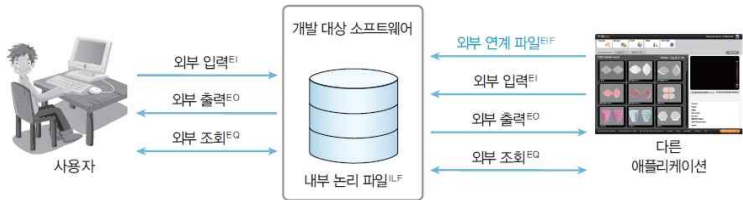


그림 3-9 사용자 관점에서 본 기능 점수의 5가지 유형

10-3 데이터 기능 점수 계산(2)

■ 내부 논리 파일ILF: Internal Logical File

- 사용자가 등록/수정/삭제/조회를 하기 위한 대상
- 데이터베이스에 존재하는 데이터 모임(데이터베이스의 정보)
- 데이터베이스 정보: 기능 점수 측정 대상으로 애플리케이션 내부에서 파일로 유지
- 데이터베이스 테이블, 시스템 내부에서 다루는 파일, 클래스 등
- 애플리케이션에 존재하는 데이터를 논리적으로 모아놓은 것
- 금번 프로젝트에서 생성하여 관리하는 데이터

■ 외부 연계 파일EIF: External Interface File

- 측정 대상 애플리케이션에서는 참조만 하고 다른 애플리케이션에서 유지되는 파일
- 다른 프로젝트에서 생성하였으나 금번 프로젝트에서 참조하는 데이터

10-3 데이터 기능 점수 계산(3)

표 3-7 외부 연계 파일과 내부 논리 파일의 예

기능	기능 유형	기능 명	기능 유형
학생 관리	학생 정보 관리	학생 기본 정보	내부 논리 파일 ^{ILF}
		학생 등록금 정보	외부 연계 파일 ^{EIF}

데이터 기능 점수 = {(ILF 개수 × 7.5) + EIF 개수 × 5.4}

7.5: 내부 논리 파일의 평균 복잡도

5.4: 외부 연계 파일의 평균 복잡도

10-4 트랜잭션 기능 점수 계산(1)

■ 트랜잭션 기능

- 입력, 출력, 조회

■ 트랜잭션 기능 측정

- 외부 입력, 외부 출력, 외부 조회의 횟수를 세는 것

■ 트랜잭션 기능 점수

- {외부 입력, 외부 출력, 외부 조회}의 개수 $\times$ 각각의 평균복잡도(가중치)

10-4 기능 유형(2)

■ 외부 입력external input

- 데이터베이스에 데이터를 등록하거나, 수정·삭제하는 것
(예) 학생 정보 등록, 수정, 삭제

■ 외부 출력external output

- 계산하는 로직을 거쳐 데이터나 제어 정보를 사용자에게 보여주는 기능
- 수학적 계산 로직이 하나 이상 존재하며 그에 따른 파생 데이터도 존재
(예) 학생 학점 조회

■ 외부 조회external inquiry

- 로직이 필요 없고 DB에 존재하는 데이터를 찾아 그대로 표시만 해주는 기능
(예) 학생 주소 검색, 학생 정보 조회

10-4 트랜잭션 기능의 예(3)

표 3-8 트랜잭션 기능의 예

기능 명	기능 유형	기능 명	기능 유형
학생 관리	학생 정보 관리	학생 정보 등록	외부 입력 ^{TI}
		학생 정보 수정	외부 입력 ^{TI}
		학생 정보 조회	외부 조회 ^{EQ}
		학생 정보 삭제	외부 입력 ^{TI}
		학생 학점 조회	외부 출력 ^{EO}
		학생 정보 테이블	내부 논리 파일 ^{ILF}
		학생 등록금 테이블	외부 연계 파일 ^{EIF}
	비밀번호 관리	비밀번호 조회	외부 조회 ^{EQ}
		비밀번호 변경	외부 입력 ^{TI}

트랜잭션 기능 점수 = {(TI개수 × 4.0) + (EO개수 × 5.2) + (EQ개수 × 3.9)}

4.0: 외부 입력의 평균 복잡도

5.2: 외부 출력의 평균 복잡도

3.9: 외부 조회의 평균 복잡도

10-5 미조정 기능 점수 계산

■ 미조정 기능 점수 UFP: Unadjusted Function Point

- 데이터 기능 점수 + 트랜잭션 기능 점수
- 단순히 기능적인 요구 사항에 대해서만 계산
- 여러 가지 특성에 대한 고려를 하지 않음

표 3-9 미조정 기능 점수의 예

기능		기능 수	평균 복잡도	기능 점수	
데이터 기능 점수	ILF	1	7.5	7.5	12.9
	EIF	1	5.4	5.4	
트랜잭션 기능 점수	EI	4	4.0	16	29
	EO	1	5.2	5.2	
	EQ	2	3.9	7.8	
계				41.9	

10-6 보정 전 개발 원가 계산

■ 보정 전 개발 원가

- 미조정 기능 점수 X 기능 점수당 단가
 - 기능 점수당 단가: 553,114원(출처: SW사업대가산정 가이드)

보정 전 개발 원가 = 미조정 기능 점수 X 기능 점수당 단가 (기능 점수당 단가: 553,114원)

- 분석/설계와 구현/테스트를 구분해 별도의 사업으로 진행 시,
복잡도 가중치와 가중치에 따른 단계별 단가를 적용

표 3-10 소프트웨어 개발 단계별 기능 점수 가중치

단계	분석	설계	구현	테스트	합계
평균 복잡도 가중치	0.19	0.24	0.32	0.25	1.00
단가	105,092원	132,747원	176,996원	138,279원	553,114원

10-7 보정 후 점수계산(1)

■ 간이 기능 점수법을 이용한 기능 점수 산정 방법

■ 보정 계수

• 보정 계수의 필요성

- 규모와 유형도 다양하고, 연계 복잡성 수준, 성능요구 수준, 운영 환경 호환성, 보안성 수준이 프로젝트의 특성에 따라 다름
- 실제 개발 비용에 영향을 미치므로 이들에 대 해 보정 계수를 정하고 이를 반영해 보정한 후 개발 원가를 구해야 함

• 규모 보정 계수

- 기능 점수가 500FP 미만이면 규모 보정 계수 1.28을, 3,000FP 초과이면 1.153을 적용

$$\text{규모 보정 계수} = 0.4057 \times (\log_{10}(\text{기능 점수}) - 7.1978)^2 + 0.8878$$

(단, 500FP 미만 시 1.2800, 3000FP 초과 시 1.1530을 적용한다.)

• 애플리케이션 복잡도 보정 계수

- 사용자가 요구하는 소프트웨어가 복잡하다면 어려운 만큼의 보정 계수를 적용
- 애플리케이션의 복잡도는 연계 복잡성 수준, 성능요구 수준, 운영 환경 호환성, 보안성 수준으로 나뉨

10-7 보정 후 점수계산(2)

■ 간이 기능 점수법을 이용한 기능 점수 산정 방법

■ 보정 계수

• 애플리케이션 복잡도 보정 계수

- 연계 복잡성 수준

» 연계 기관 수에 따른 보정 계수를 적용

표 3-11 연계 복잡성 수준에 따른 보정 계수

난이도	보정 계수
1. 타 기관과 연계 없음	0.88
2. 1~2개의 타 기관과 연계	0.94
3. 3~5개의 타 기관과 연계	1.00
4. 6~10개의 타 기관과 연계	1.06
5. 10개를 초과하는 타 기관과 연계	1.12

- 성능요구 수준

» 개발하려는 소프트웨어에 대해 사용자가 빠른 응답시간 또는 높은 처리율을 요구한다면 소프트웨어의 복잡도는 높아짐

표 3-12 성능요구 수준에 따른 보정 계수

난이도	보정 계수
1. 응답 성능에 대한 특별한 요구사항이 없다.	0.91
2. 응답 성능에 대한 요구사항이 있으나 특별한 조치는 필요하지 않다.	0.95
3. 응답시간이나 처리율이 피크(peak) 타임에 중요하여 처리 시간이 명시되어 있다.	1.00
4. 응답시간이나 처리율이 모든 업무 시간에 중요하여 처리 시간이 명시되어 있다.	1.05
5. 응답성능 요구사항이 엄격해 설계 단계부터 성능 분석이 요구되거나 설계 및 구현 단계에서 성능 분석 도구가 사용된다.	1.09

10-7 보정 후 점수계산(2)

■ 간이 기능 점수법을 이용한 기능 점수 산정 방법

■ 보정 계수

• 애플리케이션 복잡도 보정 계수

- 운영 환경 호환성

» 개발이 완료되어 설치할 때의 운영 환경(HW/SW)이 단순하지 않고 다음 표처럼 다양하다면 이를 보정 계수로 반영

표 3-13 운영 환경 호환성에 따른 보정 계수

난이도	보정 계수
1. 운영 환경 호환성에 대한 요구사항이 없다.	0.94
2. 운영 환경 호환성에 대한 요구사항이 있으며 동일 하드웨어 및 소프트웨어 환경에서 운영되도록 설계된다.	1.00
3. 유사한 운영 환경에 대한 요구사항이 있으며 유사 하드웨어 및 소프트웨어 환경에서 운영되도록 설계된다.	1.06
4. 상이한 운영 환경에 대한 요구사항이 있으며 이질적인 하드웨어 및 소프트웨어 환경에서 운영되도록 설계된다.	1.13
5. 항목 4에 대해 일관적 산출을 위해 여러 장소에서 실행한 운영을 보장하기 위한 운영 절차의 문서화와 사전 교육훈련이 요구된다.	1.19

- 보안성 수준

» 보안에 대한 요구가 많을수록 복잡도는 높아지고 이를 다음 표처럼 보정 계수로 반영

표 3-14 보안성 수준에 따른 보정 계수

난이도	보정 계수
1. 암호화 점진, 웹 취약점 점진, 시큐어 코딩, 개인정보 보호 등 1가지 보안 요구사항이 포함되어 있다.	0.97
2. 2가지 보안 요구사항이 포함되어 있다.	1.00
3. 3가지 보안 요구사항이 포함되어 있다.	1.03
4. 4가지 보안 요구사항이 포함되어 있다.	1.06
5. 5가지 이상의 보안 요구사항이 포함되어 있다.	1.08

10-9 보정 후 개발원가 계산

■ 간이 기능 점수법을 이용한 기능 점수 산정 방법

■ 보정 후 개발 원가 계산

보정 후 개발 원가 = 보정 전 개발 원가 × (규모 보정 계수 × 연계 복잡성 수준 보정 계수 × 성능요구 수준 보정 계수 × 운영 환경 호환성 보정 계수 × 보안성 수준 보정 계수)

- 5가지 보정 계수의 값을 구했다면 이 값을 적용해 보정한 후 개발 원가를 계산할 수 있음

표 3-15 기능 점수 계산을 위한 애플리케이션

애플리케이션	기능	세부 기능
학사관리	학칙 관리	학칙 자료 등록/수정/삭제/조회
	교과 과정 관리	교과 과정 등록/수정/삭제/조회 유사/동일 과목 조회
	수업 관리	개설 강좌 등록/수정/삭제/조회
	수강 관리	수강 과목 등록/수정/조회/삭제 시간표 등록/수정/삭제/조회 이수 구분 변경
	성적 관리	성적 등록/수정/조회
사용자 관리	교수 정보 관리	교수 정보 등록/수정/삭제/조회
	학생 정보 관리	학생 정보 등록/수정/삭제/조회

표 3-16 학사관리 애플리케이션에 대한 개발비 계산

단계	기능		개수	평균 가중치	기능 점수
①	데이터 기능	ILF 학칙 테이블, 교과 과정 테이블, 개설 강좌 테이블, 수강 과목 테이블, 성적 테이블	5	7.5	37.5
		EIF 교수 정보 테이블, 학생 정보 테이블	2	5.4	10.8
	트랜잭션 기능	EI 학칙 자료 등록/수정/삭제, 교과목 등록/수정/삭제, 개설 과목 등록/수정/삭제, 수강 과목 등록/수정/삭제, 이수 구분 변경, 성적 등록/수정	18	4.0	72.0
		EO 유사/동일 과목 조회, 성적 조회	2	5.2	10.4
		EQ 학칙 자료 조회, 교과 과정 조회, 개설 강좌 조회, 수강 과목 조회, 시간표 조회, 교수 정보 조회, 학생 정보 조회	7	3.9	27.3
		기능 점수의 합		158	
	②	보정 전 개발 원가		158×553,114원=87,392,012원	
③	보정 계수		규모(1.28)×연계 복잡성 수준(0.88)×성능요구 수준(0.91)×운영 환경 호환성(1.0)		
	보정 후 개발 원가		87,392,012원×1.28×0.88×0.91×1.0=89,578,909.71원		

- ① 데이터 기능과 트랜잭션 기능에 대한 기능 점수의 합(158점)을 구한다.
- ② 기능 점수(158점)에 기능 점수당 단가(553,114원)를 곱해 보정 전 개발 원가를 구한다.
- ③ 보정 전 개발 보정 계수의 값을 반영해 보정 후 개발 원가를 구한다.



Section 08 일정 계획

1. 일정 계획



그림 3-10 프로젝트 일정 계획의 예

1-1 SW 개발 프로젝트에서의 일정 계획

■ 일정 계획

- 작업 순서 결정, 소작업의 개발 기간, 순서, 필요한 자원 등의 일정 계획

표 3-16 일정 계획

총 개발 기간	6개월(○○○○. 1. 1. ~ ○○○○. 6. 31.)															
소작업	교과 과정 관리, 수업 관리, 수강 관리, 성적 관리, 사용자 정보 관리															
소작업별 개발 기간	<table><tr><td>소작업</td><td>개발 기간</td></tr><tr><td>사용자 정보 관리</td><td>1개월(1. 1. ~ 1. 31.)</td></tr><tr><td>교과 과정 관리</td><td>1개월(2. 1. ~ 2. 28.)</td></tr><tr><td>수업 관리</td><td>1개월(3. 1. ~ 3. 31.)</td></tr><tr><td>수강 관리</td><td>1개월(4. 1. ~ 4. 30.)</td></tr><tr><td>성적 관리</td><td>1개월(5. 1. ~ 5. 31.)</td></tr><tr><td>테스트</td><td>1개월(6. 1. ~ 6. 30.)</td></tr></table>		소작업	개발 기간	사용자 정보 관리	1개월(1. 1. ~ 1. 31.)	교과 과정 관리	1개월(2. 1. ~ 2. 28.)	수업 관리	1개월(3. 1. ~ 3. 31.)	수강 관리	1개월(4. 1. ~ 4. 30.)	성적 관리	1개월(5. 1. ~ 5. 31.)	테스트	1개월(6. 1. ~ 6. 30.)
	소작업	개발 기간														
	사용자 정보 관리	1개월(1. 1. ~ 1. 31.)														
	교과 과정 관리	1개월(2. 1. ~ 2. 28.)														
	수업 관리	1개월(3. 1. ~ 3. 31.)														
	수강 관리	1개월(4. 1. ~ 4. 30.)														
	성적 관리	1개월(5. 1. ~ 5. 31.)														
	테스트	1개월(6. 1. ~ 6. 30.)														
개발 순서	사용자 정보 관리 → 교과 과정 관리 → 수업 관리 → 수강 관리 → 성적 관리															
필요 자원	개발 툴(파워빌더), 개발용 PC, 개발 공간															

2. 작업 분할 구조도(1)

■ 작업 분할 구조도(WBS: Work Breakdown Structure)

- 프로젝트 목표를 달성하기 위해 필요한 활동과 업무를 세분화하는 작업
- 프로젝트 구성 요소들을 계층 구조로 분류
- 프로젝트의 전체 범위 정의
- 프로젝트 작업을 세분화

■ 작업 패키지(work package)

- 계층 구조에서 최하위에 있는 항목

2. 작업 분할 구조도(2)

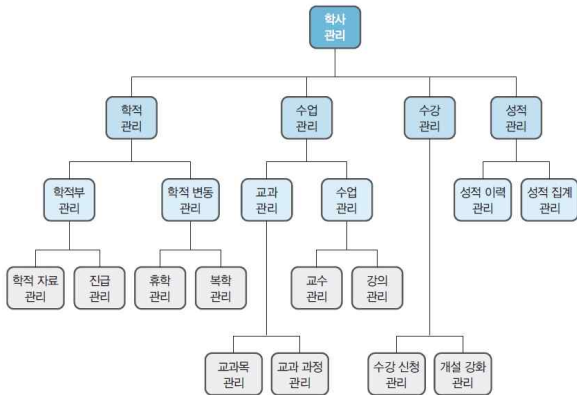


그림 3-11 학사 관리 시스템의 WBS

2. 작업 분할 구조도(3)

■ 작업 분할 구조도의 목적과 용도

- 사용자와 개발자 간의 의사소통 도구로 사용함.
- 프로젝트 업무 내역을 가시화할 수 있어 관리가 용이함.
- 프로젝트 팀원의 책임과 역할이 분명함.
- 필요 인력과 일정 계획을 세우는 데 기초로 활용함.
- 개발비 산정 시 기초로 활용함.
- 성과 측정 및 조정 시 기준선으로 활용할 수 있음.

3. 네트워크 차트

■ PERT/CPM(Program Evaluation and Review Technique/Critical Path Method)

- WBS의 작업 순서, 소요 기간 등을 네트워크 형태의 그래프로 표현한 후 어떤 작업이 중요한지, 또 일정에 여유가 있는 작업은 어떤 것인지 찾아내 중점 관리를 해야 하는 작업을 명확히 하는데 사용
- 프로젝트를 완료할 수 있는 최소 기간은 얼마인지, 완료 기간을 맞추기 위해서는 각 작업을 언제 시작하고 완료해야 하는지, 지연되지 않으려면 어떤 작업에 특히 주의를 기울여야 하는지, 또 전체 프로젝트 완료 기간을 단축하기 위해서는 어떤 작업들을 단축하는 것이 가장 경제적인지 등 관리자의 고민에 답을 주기 위해 필요한 도구

4. 학사 관리 애플리케이션에 대한 CPM의 예

표 3-17 CPM 네트워크를 위한 활동 목록

작업	작업 설명	선행 작업	소요 기간(주)
A	개인 정보 등록/수정/조회/삭제 프로그램 개발	-	2
B	학적 변동 자료 등록/수정/조회/삭제 프로그램 개발	A	6
C	휴·복학 및 자퇴 등록/수정/조회/삭제 프로그램 개발	A	4
D	교육 과정 등록/수정/조회/삭제 프로그램 개발	B, C	2
E	유사/동일 과목 등록/수정/조회/삭제 프로그램 개발	D	4
F	개설 강좌 등록/수정/조회/삭제 프로그램 개발	D	3
G	수강 과목 등록/수정/조회/삭제 프로그램 개발	E	2
H	시간표 등록/수정/조회/삭제 프로그램 개발	E	4
I	성적 등록/수정/조회/삭제 프로그램 개발	F	2
J	장학생 등록/수정/조회/삭제 프로그램 개발	F	1
K	등록금 등록/수정/조회/삭제 프로그램 개발	G, H	2
L	졸업 사정 등록/수정/조회/삭제 프로그램 개발	I, K	2
M	사회봉사 실적 등록/수정/조회/삭제 프로그램 개발	J, L	3

① CPM 네트워크를 그린다

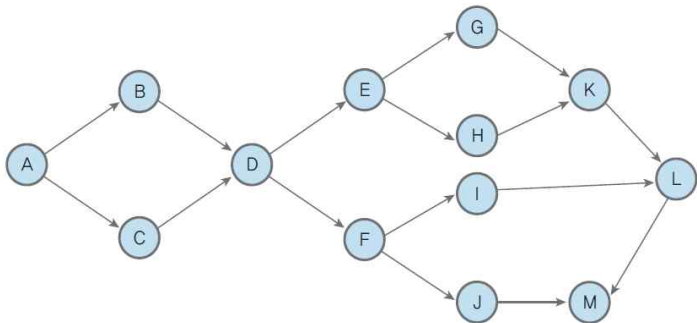


그림 3-12 CPM 네트워크

② ES값을 구한다

■ ES Earliest Start Time

- 가능한 빨리 시작할 수 있는 시간으로, 선행 작업이 완료되었을 때 해당작업을 시작할 수 있는 가장 빠른 시점
- ES 값을 구할 때는 맨 앞(작업 A)에서 끝 방향으로 가며 계산
- 두 작업이 합류하는 지점의 경우, 이 작업의 시작 시간은 두 작업이 완전히 끝났을 때를 시작 시간으로 봄

표 3-18 작업별 가장 빨리 시작할 수 있는 시간 (단위: 주)

작업	A	B	C	D	E	F	G	H	I	J	K	L	M
작업 시작 시간	0	2	2	8	10	10	14	14	13	13	18	20	22
작업 시간	2	6	4	2	4	3	2	4	2	1	2	2	3

③ EF값을 구한다

■ EF Earliest Finish Time

- 가장 빠른 시작 시간(ES)으로 시작했을 때의 가장 빠른 완료 시간
(즉) 가능한 빨리 끝낼 수 있는 시간으로 'ES+작업 소요 시간'

표 3-19 작업별 가장 빨리 완료할 수 있는 시간 (단위: 주)

작업	A	B	C	D	E	F	G	H	I	J	K	L	M
EF	2	8	6	10	14	13	16	18	15	14	20	22	25

④ LS값을 구한다

■ LS Latest Start Time

- 어떤 작업을 늦어도 시작해야 하는 시간, 즉 가장 늦게 시작할 수 있는 시간
- 이 시간에 시작하지 않으면(이 시간보다 늦게 시작하면) 총 일정이 지연
- 맨 뒤(작업 M)에서 앞 (작업 A) 방향으로 가며 계산

표 3-20 작업별 가장 늦게 시작할 수 있는 시간(단위: 주)

작업	A	B	C	D	E	F	G	H	I	J	K	L	M
작업 시작 시간	0	2	4	8	10	15	16	14	18	21	18	20	22
작업 시간	2	6	4	2	4	3	2	4	2	1	2	2	3

⑤ LF값을 구한다

■ LF Latest Finish Time

- 가장 늦게 시작할 수 있는 시간(LS)에 시작해 작업을 완료한 시간
(즉) 작업을 가장 늦게 끝낼 수 있는 시간으로 'LS+작업 소요 시간'

표 3-21 작업별 가장 늦게 시작했을 때 완료할 수 있는 시간 (단위: 주)

작업	A	B	C	D	E	F	G	H	I	J	K	L	M
작업 시작 시간	0	2	4	8	10	15	16	14	18	21	18	20	22
작업 시간	2	6	4	2	4	3	2	4	2	1	2	2	3
작업 완료 시간	2	8	8	10	14	18	18	18	20	22	20	22	25

⑥ ST값을 구한다

■ STSlack Time

- '느슨한 시간', '여유 있는 시간'
- 여유 시간 = 가장 늦게 시작하는 시간 - 가장 빨리 시작하는 시간

표 3-22 여유 시간(단위: 주)

작업	A	B	C	D	E	F	G	H	I	J	K	L	M
작업별 빠른 시작 시간	0	2	2	8	10	10	14	14	13	13	18	20	22
작업별 늦은 시작 시간	0	2	4	8	10	15	16	14	18	21	18	20	22
여유 시간	0	0	2	0	0	5	2	0	5	8	0	0	0

⑦ 임계 경로를 구한다

■ 임계 경로 critical path

- [그림 3-13]의 그래프에서 여유 시간이 없는 경로

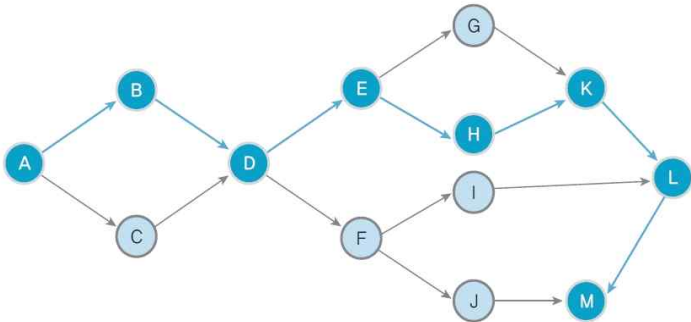


그림 3-13 CPM 네트워크와 임계 경로

5. 간트 차트를 이용한 일정표 작성

■ 간트 차트 Gantt chart

- 프로젝트 일정 관리를 위한 바^{bar} 형태의 도구



그림 3-14 간트 차트를 이용한 프로젝트 일정 계획표



Section 09 위험 분석

1. 위험 분석



그림 3-15 위험 예방



그림 3-16 도구를 사용한 위험 예방

2. 위험 예방



그림 3-17 위험 예방을 위한 도구 준비

3. 위험 관리 절차(1)

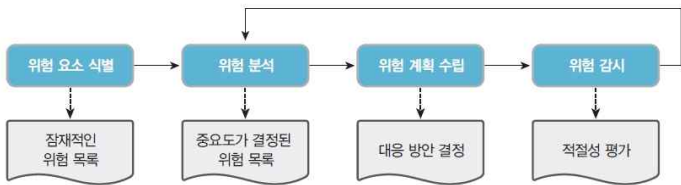


그림 3-18 위험 관리 절차

3. 위험 관리 절차(2)

■ 위험 요소 식별

표 3-23 프로젝트 수행 시 발생할 수 있는 위험 요소

위험 요소	위험 내용
개발자의 이직	프로젝트 수행 중 개발자들이 이직한다.
요구 사항 변경	요구 사항 확정 이후에도 변경 요구가 계속된다.
발주사의 재정적 어려움	프로젝트 수행 중 고객사에 경제적인 어려움이 발생한다.
예상을 빚나간 투입 인력	처음에 예측한 인력보다 인력이 더 많이 필요하다.
개발 기간 부족	처음에 예측한 개발 기간을 초과한다.
개발비 초과	처음에 예측한 개발비로 개발을 완료할 수 없다.

3. 위험 관리 절차(3)

■ 위험 분석

- 위험 요소가 발생할 가능성과 영향력을 판단
- 과거 프로젝트에서 데이터와 위험을 분석한 경험이 많은 개발자에 의존해 판단
- 위험 발생 가능성의 척도
 - '매우 낮음(<10%), 낮음(10~25%), 보통(25~50%), 높음(50~75%), 매우 높음(>75%)'
- 위험 발생 확률
 - 상: 80% 이상, 중: 30~80%, 하: 30% 미만
- 영향력
 - 재앙, 심각함, 해결 가능함, 경미함 등으로 분류
 - 비용과 일정으로 분류: 상(20% 이상 초과), 중(5~20%), 하(5% 이하)
- 비용과 일정
 - 상(20% 이상 초과), 중(5~20%), 하(5% 이하)

3. 위험 관리 절차(4)

■ 위험 계획 수립

- 식별된 위험 요소의 위험을 관리하기 위해 전략을 찾는 과정
- 위험을 처리하는 위험 대응 방안 수립

■ 위험 감시

- 식별된 위험 요소의 발생 확률과 변화 등을 관리
- 예측한 위험의 실제 발생 여부 확인
- 실전에서 위험 대응 방안의 적절성 여부 확인



Thank You
